Assignment_15.1

P.Naga Shiva Chaitanya

2303A51945 B - 27

Task 1 – Student Records API

Task: Use AI to build a RESTful API for managing student records.

Instructions:

• Endpoints required:

o GET /students → List all students

o POST /students → Add a new student

o PUT /students/{id} → Update student details

o DELETE /students/{id} → Delete a student record

• Use an in-memory data structure (list or dictionary) to store records.

• Ensure API responses are in JSON format.

Expected Output:

• Working API with CRUD functionality for student records.

PROMPT:

"Create a Flask REST API for student records with CRUD operations using in-memory storage. Return JSON responses and include comments."

CODE:

```
from flask import Flask, jsonify, request


app = Flask(__name__)


students = []

next_id = 1


def find_student(student_id):

    for s in students:

        if s["id"] == student_id:

            return s

    return None


@app.route("/students", methods=["GET"])

def get_students():

    return jsonify({"students": students}), 200


@app.route("/students", methods=["POST"])

def add_student():

    global next_id

    data = request.get_json()
```

```
    if not data or "name" not in data or "age" not in data or "course" not
in data:

        return jsonify({"error": "Missing fields"}), 400



    student = {"id": next_id, "name": data["name"], "age": data["age"],
"course": data["course"]}

    students.append(student)

    next_id += 1



    return jsonify({"message": "Student added", "student": student}), 201



@app.route("/students/<int:id>", methods=["PUT"])

def update_student(id):

    s = find_student(id)

    if not s:

        return jsonify({"error": "Not found"}), 404

    data = request.get_json()

    s.update(data)

    return jsonify({"message": "Updated", "student": s}), 200

@app.route("/students/<int:id>", methods=["DELETE"])

def delete_student(id):

    s = find_student(id)

    if not s:

        return jsonify({"error": "Not found"}), 404
```

```python
    students.remove(s)

    return jsonify({"message": "Deleted"}), 200

if __name__ == "__main__":

    with app.test_client() as c:

        print(c.get("/students").json)

        print(c.post("/students",
json={"name":"A","age":20,"course":"CSE"}).json)

        print(c.put("/students/1",
json={"name":"B","age":21,"course":"IT"}).json)

        print(c.delete("/students/1").json)



    app.run(debug=True)
```

OUTPUT:

```
PS C:\Users\nagas\OneDrive\Desktop\bobby\python> & C:/Python314/python.exe c:/Users/nagas/OneDrive/Desktop/bobby/python/student_api.py
{'students': []}
{'message': 'Student added', 'student': {'age': 20, 'course': 'CSE', 'id': 1, 'name': 'A'}}
{'message': 'Updated', 'student': {'age': 21, 'course': 'IT', 'id': 1, 'name': 'B'}}
{'message': 'Deleted'}
 * Serving Flask app 'student_api'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
{'students': []}
{'message': 'Student added', 'student': {'age': 20, 'course': 'CSE', 'id': 1, 'name': 'A'}}
{'message': 'Updated', 'student': {'age': 21, 'course': 'IT', 'id': 1, 'name': 'B'}}
{'message': 'Deleted'}
 * Debugger is active!
 * Debugger PIN: 733-084-019
```

Task 2 – Library Book Management API

Task: Develop a RESTful API to handle library books.

Instructions:

• Endpoints required:

o GET /books → Retrieve all books

o POST /books → Add a new book

o GET /books/{id} → Get details of a specific book

o PATCH /books/{id} → Update partial book details

(e.g., availability)

o DELETE /books/{id} → Remove a book

• Implement error handling for invalid requests.

Expected Output:

• Functional API with CRUD + partial updates.

OUTPUT:

```
from flask import Flask, jsonify, request



app = Flask(__name__)



books = []

next_id = 1



def find_book(id):

    for b in books:
```

```
        if b["id"] == id:
            return b
    return None


@app.route("/books", methods=["GET"])
def get_books():
    return jsonify({"books": books}), 200


@app.route("/books", methods=["POST"])
def add_book():
    global next_id
    data = request.get_json()

    if not data or "title" not in data or "author" not in data:
        return jsonify({"error": "Missing fields"}), 400

    book = {"id": next_id, "title": data["title"], "author":
data["author"], "available": True}
    books.append(book)
    next_id += 1

    return jsonify({"message": "Added", "book": book}), 201


@app.route("/books/<int:id>", methods=["GET"])
```

```python
def get_book(id):
    b = find_book(id)
    if not b:
        return jsonify({"error": "Not found"}), 404
    return jsonify(b), 200


@app.route("/books/<int:id>", methods=["PATCH"])
def update_book(id):
    b = find_book(id)
    if not b:
        return jsonify({"error": "Not found"}), 404


    b.update(request.get_json())
    return jsonify({"message": "Updated", "book": b}), 200


@app.route("/books/<int:id>", methods=["DELETE"])
def delete_book(id):
    b = find_book(id)
    if not b:
        return jsonify({"error": "Not found"}), 404


    books.remove(b)
    return jsonify({"message": "Deleted"}), 200
```

```python
if __name__ == "__main__":

    with app.test_client() as c:

        print(c.post("/books",
json={"title":"Python","author":"John"}).json)

        print(c.get("/books").json)

        print(c.patch("/books/1", json={"available":False}).json)

        print(c.delete("/books/1").json)



    app.run(debug=True)
```

OUTPUT:

```
PS C:\Users\nagas\OneDrive\Desktop\bobby\python> & C:/Python314/python.exe c:/Users/nagas/OneDrive/Desktop/bobby/python/book_api.py
{'book': {'author': 'John', 'available': True, 'id': 1, 'title': 'Python'}, 'message': 'Added'}
{'books': [{'author': 'John', 'available': True, 'id': 1, 'title': 'Python'}]}
{'book': {'author': 'John', 'available': False, 'id': 1, 'title': 'Python'}, 'message': 'Updated'}
{'message': 'Deleted'}
 * Serving Flask app 'book_api'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
{'book': {'author': 'John', 'available': True, 'id': 1, 'title': 'Python'}, 'message': 'Added'}
{'books': [{'author': 'John', 'available': True, 'id': 1, 'title': 'Python'}]}
{'book': {'author': 'John', 'available': False, 'id': 1, 'title': 'Python'}, 'message': 'Updated'}
{'message': 'Deleted'}
 * Debugger is active!
 * Debugger PIN: 733-084-019
```

Task 3 – Employee Payroll API

Task:

Create an API for managing employees and their salaries.

Instructions:

• Endpoints required:

o GET /employees → List all employees

o POST /employees → Add a new employee with

salary details

o PUT /employees/{id}/salary → Update salary of an

employee

o DELETE /employees/{id} → Remove employee

from system

• Use AI to:

o Suggest data model structure.

o Add comments/docstrings for all endpoints.

Expected Output:

• API supporting salary management with clear

documentation.

CODE:

```
from flask import Flask, jsonify, request
```

```python
app = Flask(__name__)


employees = []

next_id = 1


def find_emp(id):

    for e in employees:

        if e["id"] == id:

            return e

    return None


@app.route("/employees", methods=["GET"])

def get_emp():

    return jsonify({"employees": employees}), 200


@app.route("/employees", methods=["POST"])

def add_emp():

    global next_id

    data = request.get_json()


    if not data or "name" not in data or "role" not in data or "salary"
not in data:

        return jsonify({"error": "Missing fields"}), 400
```

```
    emp = {"id": next_id, "name": data["name"], "role": data["role"],
"salary": data["salary"]}

    employees.append(emp)

    next_id += 1



    return jsonify({"message": "Added", "employee": emp}), 201


@app.route("/employees/<int:id>/salary", methods=["PUT"])

def update_salary(id):

    e = find_emp(id)

    if not e:

        return jsonify({"error": "Not found"}), 404


    e["salary"] = request.get_json().get("salary")

    return jsonify({"message": "Updated", "employee": e}), 200


@app.route("/employees/<int:id>", methods=["DELETE"])

def delete_emp(id):

    e = find_emp(id)

    if not e:

        return jsonify({"error": "Not found"}), 404


    employees.remove(e)

    return jsonify({"message": "Deleted"}), 200
```

```python
if __name__ == "__main__":
    with app.test_client() as c:
        print(c.post("/employees",
json={"name":"A","role":"Dev","salary":5000}).json)
        print(c.put("/employees/1/salary", json={"salary":6000}).json)
        print(c.delete("/employees/1").json)

    app.run(debug=True)
```

OUTPUT:

```
PS C:\Users\nagas\OneDrive\Desktop\bobby\python> & C:/Python314/python.exe c:/Users/nagas/OneDrive/Desktop/bobby/python/employee_api.py
{'employee': {'id': 1, 'name': 'A', 'role': 'Dev', 'salary': 5000}, 'message': 'Added'}
{'employee': {'id': 1, 'name': 'A', 'role': 'Dev', 'salary': 6000}, 'message': 'Updated'}
{'message': 'Deleted'}
 * Serving Flask app 'employee_api'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
{'employee': {'id': 1, 'name': 'A', 'role': 'Dev', 'salary': 5000}, 'message': 'Added'}
{'employee': {'id': 1, 'name': 'A', 'role': 'Dev', 'salary': 6000}, 'message': 'Updated'}
{'message': 'Deleted'}
 * Debugger is active!
 * Debugger PIN: 733-084-019
```

Task 4 – Real-Time Application: Online Food Ordering API

Scenario:

Design a simple API for an online food ordering system.

Requirements:

• Endpoints required:

o GET /menu → List available dishes

o POST /order → Place a new order

o GET /order/{id} → Track order status

o PUT /order/{id} → Update an existing order (e.g., change items)

o DELETE /order/{id} → Cancel an order

• AI should generate:

o REST API code

o Suggested improvements (like authentication, pagination)

Expected Output:

• Fully working API simulating a food ordering backend.

 Deliverables (For All Tasks)

1. AI-generated prompts for code and test case generation.

2. At least 3 assert test cases for each task.

3. AI-generated initial code and execution screenshots.

4. Analysis of whether code passes all tests.

5. Improved final version with inline comments and explanation.

6. Compiled report (Word/PDF) with prompts, test cases, assertions, code, and output.

CODE:

```
from flask import Flask, jsonify, request


app = Flask(__name__)


menu = {"pizza":200, "burger":100, "pasta":150}

orders = []

next_id = 1


def find_order(id):

    for o in orders:

        if o["id"] == id:

            return o

    return None


def calc(items):

    total = 0

    for i in items:

        if i not in menu:

            return None

        total += menu[i]

    return total
```

```python
@app.route("/menu", methods=["GET"])
def get_menu():
    return jsonify(menu), 200


@app.route("/order", methods=["POST"])
def place():
    global next_id
    data = request.get_json()


    total = calc(data["items"])
    if total is None:
        return jsonify({"error":"Invalid item"}), 400


    order = {"id":next_id,"items":data["items"],"total":total}
    orders.append(order)
    next_id += 1


    return jsonify(order), 201


@app.route("/order/<int:id>", methods=["GET"])
def get_order(id):
    o = find_order(id)
    if not o:
```

```
        return jsonify({"error":"Not found"}), 404
    return jsonify(o), 200


@app.route("/order/<int:id>", methods=["PUT"])
def update(id):
    o = find_order(id)
    if not o:
        return jsonify({"error":"Not found"}), 404


    total = calc(request.get_json()["items"])
    o["items"] = request.get_json()["items"]
    o["total"] = total


    return jsonify(o), 200


@app.route("/order/<int:id>", methods=["DELETE"])
def delete(id):
    o = find_order(id)
    if not o:
        return jsonify({"error":"Not found"}), 404


    orders.remove(o)
    return jsonify({"message":"Deleted"}), 200
```

```python
if __name__ == "__main__":
    with app.test_client() as c:
        print(c.post("/order", json={"items":["pizza"]}).json)
        print(c.get("/order/1").json)
        print(c.delete("/order/1").json)

    app.run(debug=True)
```

OUTPUT:

```
PS C:\Users\nagas\OneDrive\Desktop\bobby\python> & C:/Python314/python.exe c:/Users/nagas/OneDrive/Desktop/bobby/python/food_api.py
{'id': 1, 'items': ['pizza'], 'total': 200}
{'id': 1, 'items': ['pizza'], 'total': 200}
{'message': 'Deleted'}
 * Serving Flask app 'food_api'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
{'id': 1, 'items': ['pizza'], 'total': 200}
{'id': 1, 'items': ['pizza'], 'total': 200}
{'message': 'Deleted'}
 * Debugger is active!
 * Debugger PIN: 733-084-019
127.0.0.1 - - [23/Mar/2026 10:49:13] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [23/Mar/2026 10:49:14] "GET /favicon.ico HTTP/1.1" 404 -
```